



UT-4

Programación de aplicaciones Android View Binding



¿Qué es View Binding?

- **View Binding** es una forma segura y sencilla de acceder a las vistas (**TextView**, **Button**, etc.) definidas en nuestros archivos XML sin usar `findViewById`.
- Android Studio genera automáticamente una clase de binding por cada layout XML. Esa clase contiene referencias directas a todas las vistas con id.



Ventajas de usar View Binding

- Código más limpio
- Type-safe (el compilador detecta errores)
- Más rápido de escribir



Activar View Binding

- En el archivo **build.gradle (Module)** tenemos que añadir en el apartado **android**:

```
buildFeatures {  
    viewBinding = true  
}
```



¿Qué genera Android?

- Si tenemos un layout llamado:

activity_main.xml

- Android genera automáticamente una clase:

ActivityMainBinding



¿Qué genera Android?

- Cada vista del diseño que tenga el atributo android:id se convierte en una propiedad:

```
<TextView  
    android:id="@+id/tvTitle"  
    android:layout_width="wrap_content"  
    android:layout_height="wrap_content" />
```

- En la variable que contenga el binding tendremos:

binding.tvTitle



Ejemplo

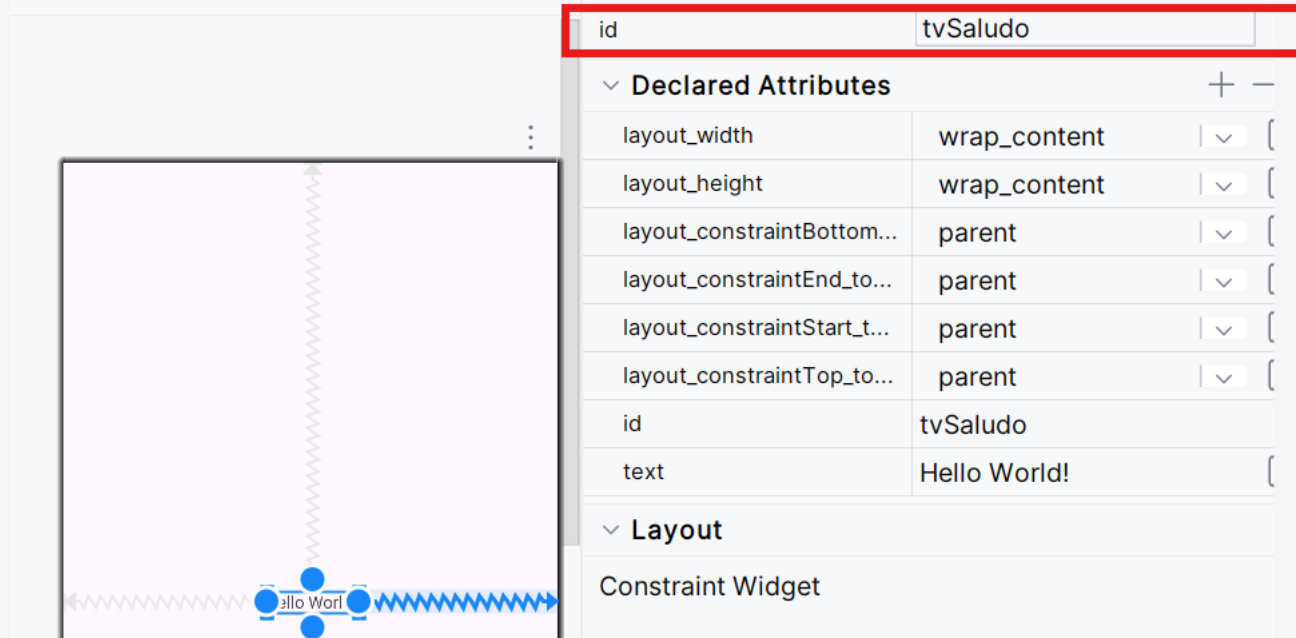
- Crea un nuevo proyecto que se llame **ViewBinding**
- Añadimos la dependencia en el archivo **build.gradle (Module)**

```
5      android {  
6  
7          buildFeatures {  
8              viewBinding = true  
9          }  
10     }
```



Ejemplo

- El layout por defecto se llama activity_main.xml y tiene un TextView con el texto “Hello World” y no tiene id
- Le asignamos un id al TextView:





Ejemplo

- La clase MainActivity tiene ahora mismo este aspecto:

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
  
        setContentView(R.layout.activity_main)  
  
        ViewCompat.setOnApplyWindowInsetsListener( view = findViewById( id = R.id.main)) { v, insets ->  
            ...  
        }  
    }  
}
```



Ejemplo

- Para crear la variable de binding sustituimos:

```
setContentView(R.layout.activity_main)
```

- Por:

```
val binding = ActivityMainBinding.inflate(layoutInflater)  
setContentView(binding.root)
```

- **inflate(layoutInflater)** crea el binding
- `binding.root` es la vista raíz del layout



Ejemplo

- La clase queda así:

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
  
        val binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        ViewCompat.setOnApplyWindowInsetsListener(view = findViewById(id = R.id.main)) { v, insets ->  
            // ...  
        }  
    }  
}
```



Ejemplo

- Mediante la variable binding podemos acceder a las vistas:

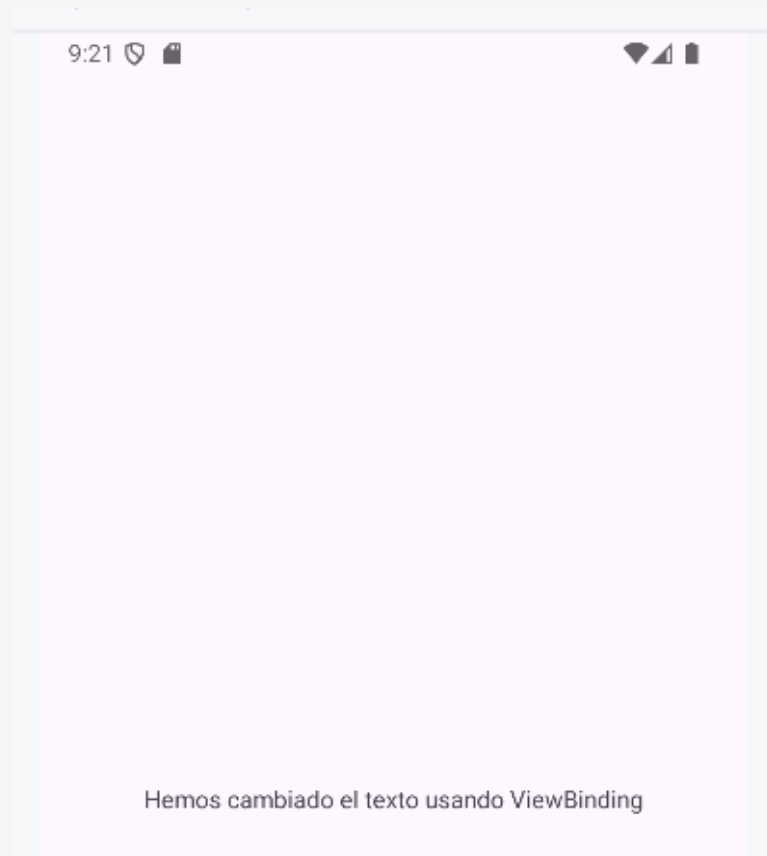
```
val binding = ActivityMainBinding.inflate(layoutInflater)  
setContentView(binding.root)
```

```
binding.tvSaludo.text="Hemos cambiado el texto usando ViewBinding"
```



Ejemplo

- Si ejecutamos el proyecto:





Ejemplo 2

- En el mismo proyecto vamos a crear un nuevo layout que se llame:

layout_prueba_viewbinding

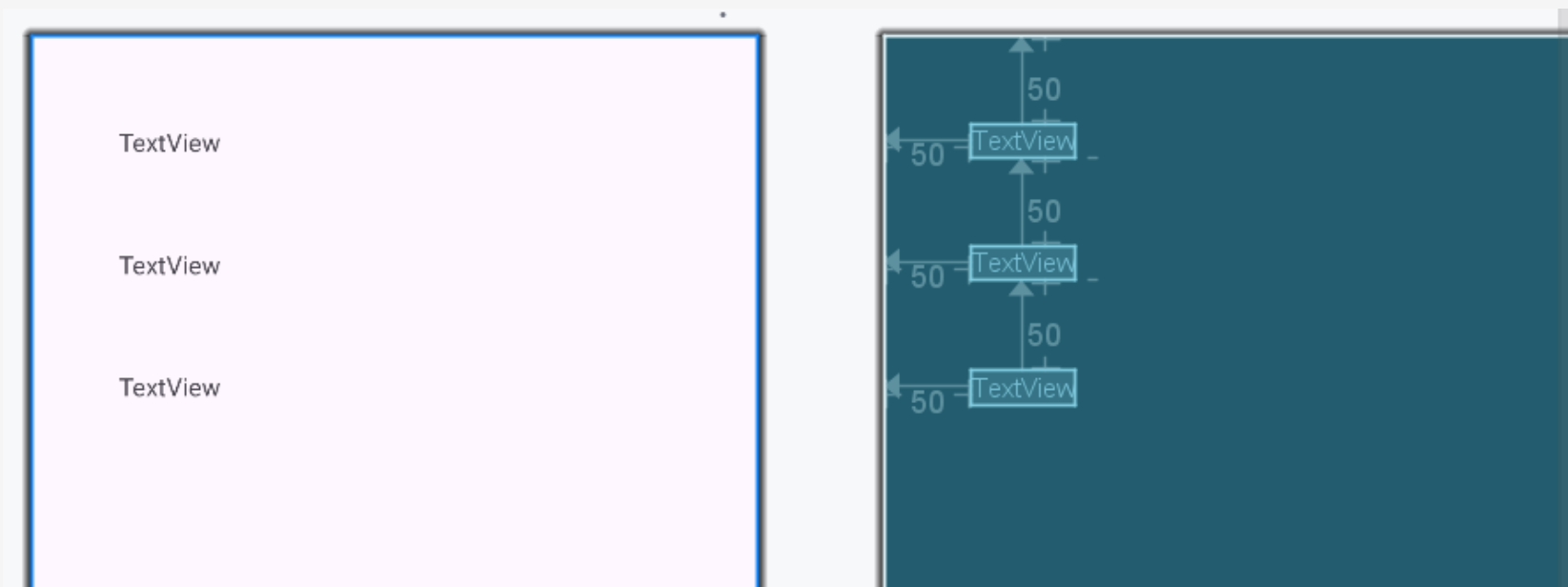
New Resource File

File name:	layout_prueba_viewbinding
Root element:	androidx.constraintlayout.widget.ConstraintLayout
Source set:	main <i>src/main/res</i>



Ejemplo 2

- Añadimos 3 TextView al diseño:





Ejemplo 2

- Les asignamos los id:
 - tvUno
 - tvDos
 - tvTres
- Le asignamos un id al constraintLayout:
 - second

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/second"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```



Ejemplo 2

- Actualizamos MainActivity

```
val binding = LayoutPruebaViewbindingBinding.inflate(LayoutInflater)
```

- Como el nuevo layout se llama **layout_prueba_viewbinding** android ha generado la clase **LayoutPruebaViewbindingBinding** es decir ha cogido cada trozo del nombre del layout separado por un guión y lo ha puesto con la primera letra en mayúsculas y ha añadido al final **Binding**
- layout -> **Layout**
- prueba -> **Prueba**
- viewBinding -> **ViewBinding**
- **Layout + Prueba + Viewbinding + Binding = LayoutPruebaViewbindingBinding**



Ejemplo 2

```
val binding = LayoutPruebaViewbindingBinding.inflate(layoutInflater)
setContentView(binding.root)
```

```
binding.tvUno.text="Texto 1"
```

```
binding.tvDos.text="Texto 2"
```

```
binding.tvTres.text="Texto 3"
```



Ejemplo 2

- También tenemos que actualizar el nombre del constraint layout:

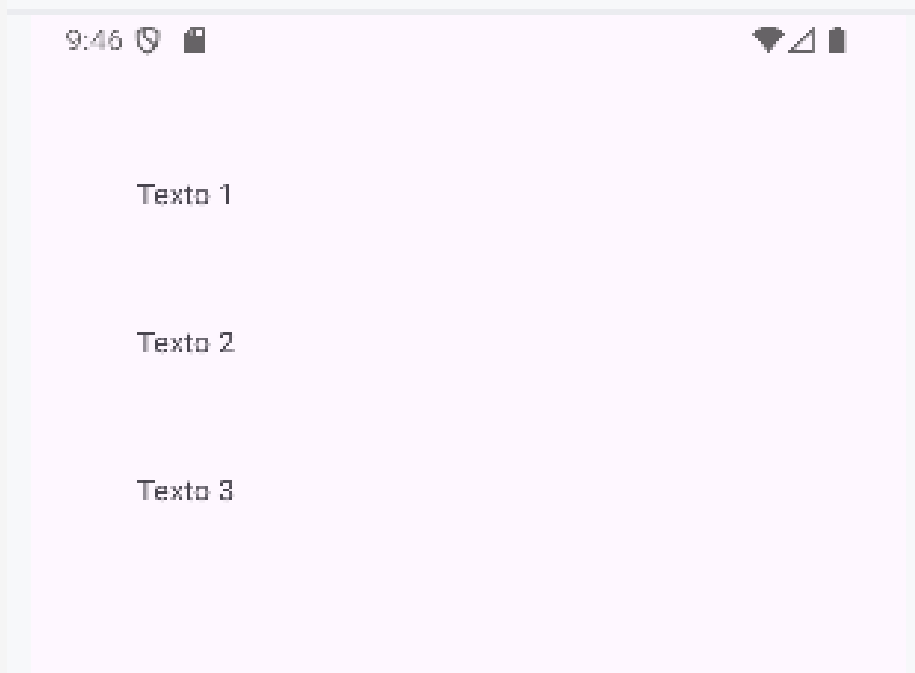
```
ViewCompat.setOnApplyWindowInsetsListener( view = findViewById( id = R.id.second)) { v, insets ->
    val systemBars = insets.getInsets( typeMask = WindowInsetsCompat.Type.systemBars())
    v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
    insets
}
```

- Este código ajusta el padding de una vista para que no quede debajo de la barra de estado o la barra de navegación.



Ejemplo 2

- Si ejecutamos el proyecto:





Ejemplo 3

- Uso de ViewBinding en el diálogo personalizado que vimos en los diálogos

```
val builder = AlertDialog.Builder( context = this)
// Obtenemos el inflater.
val inflater = LayoutInflater;
// Creamos el binding
val binding = DialogoLoginBinding.inflate(LayoutInflater)

builder.setView(binding.root)

// Añadimos los botones
.setPositiveButton( text = "Acceder") { dialog, id ->
    // Código para validar al usuario
    //val etNombre = binding.bt
    // etContra: EditText=dialogView.findViewById(R.id.contraseña)
    if(binding.usuario.text.toString()=="paco"&&binding.contraseA.text.toString()=="1234")
        Toast.makeText( context = this, text = "Login correcto", duration = Toast.LENGTH_LONG).show()
    else
        Toast.makeText( context = this, text = "Login incorrecto", duration = Toast.LENGTH_LONG).show()
}

.setNegativeButton( text = "Cancelar") { dialog, id ->
```